

Firmware de baixo nível para a mão HACKberry — e como se faz isso pelo VS Code

Um documento didático em duas metades. A primeira explica **como o firmware foi pensado**: as decisões de baixo nível, na ordem em que apareceram, e o raciocínio que produziu cada uma. A segunda mostra **o fluxo de trabalho real** — Arduino sem a IDE do Arduino, tudo pelo VS Code e pela linha de comando.

Escrito para quem nunca programou um microcontrolador. Todo comando aqui é o comando que este projeto usa de verdade; todo número vem do código, não de exemplo.

Alvo: Arduino Nano clone (ATmega328P, CH340) Firmware: 260 linhas, C++ / GPLv3

firmware/hackberry_serial/hackberry_serial.ino

Parte 1

O terreno: programar sem sistema operacional

Parte 3

Anatomia do `loop()`, na ordem de execução

Parte 5

O fluxo no VS Code, do zero ao servo girando

Parte 2

A decisão que veio antes do código: o protocolo

Parte 4

Seis decisões de baixo nível, e o porquê de cada uma

Parte 6

Como se prova que funcionou

PARTE 1

O terreno: programar sem sistema operacional

No PC, quando um programa demora, o sistema operacional tira ele da frente e deixa outro rodar. No ATmega328P **não existe sistema operacional**. Existe um único programa, sozinho no chip, e ele é responsável por absolutamente tudo — inclusive por não travar. Essa é a diferença que organiza todas as decisões deste firmware.

01

Um programa, para sempre

`setup()` roda uma vez ao ligar; `loop()` roda em seguida, infinitamente, milhões de vezes por segundo. Não há “fim do programa”.

02

Recursos que caberiam num e-mail

16 MHz, 32 KB de flash para o código, **2 KB de RAM**. Um buffer de texto generoso aqui tem 48 bytes — e é o que o firmware usa.

03

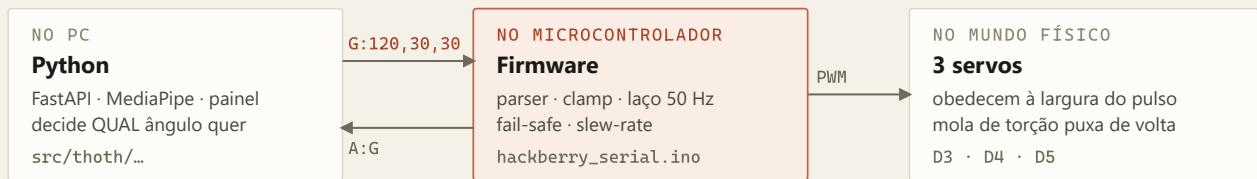
Nada pode bloquear

Enquanto o programa espera, o mundo não espera. Um `delay(500)` significa meio segundo cego: comandos perdidos, fail-safe surdo, servo sem atualização.

04

O único canal é texto

O PC e o chip conversam por uma linha serial a 115200 bps. Uma linha de texto terminada em `\n` vai; outra volta. É só isso.



cabo USB · serial 115200 8N1 · uma linha de texto terminada em \n

pulso 1-2 ms a cada 20 ms

O firmware é a única peça entre uma intenção em Python e um torque real. Tudo que ele deixa passar sem validar chega ao servo — por isso ele não confia no host (Parte 4, decisão 3).

Três mundos, dois cabos, um protocolo de texto. O PC decide o *quê*; o firmware decide *como* e *se*.

Por que `delay()` é veneno

Quase todo tutorial de Arduino começa com `delay()`. Ele funciona para piscar um LED e falha em qualquer coisa que precise reagir. A diferença entre as duas versões abaixo é a diferença entre um brinquedo e um controlador.

```
// RUIM - jeito ingênuo: o chip fica cego por 500 ms em cada passo
servo.write(alvo);
delay(500);           // nesses 500 ms: serial ignorada, fail-safe surdo

// BOM - jeito do firmware: consulta o relógio e volta na próxima volta
if (millis() - lastTick >= TICK_MS) { // TICK_MS = 20 → 50 Hz
    lastTick = millis();
    controlTick();                     // dá UM passo e devolve o controle
}
```

A IDEIA CENTRAL

`millis()` devolve quantos milissegundos passaram desde que a placa ligou. Em vez de *esperar*, o programa *pergunta que horas são* e decide se já é hora de agir. Assim uma volta do `loop()` dura microssegundos, e as três tarefas do firmware — ler serial, vigiar o heartbeat, mover o servo — parecem simultâneas sem que exista nenhuma thread.

PARTE 2

A decisão que veio antes do código: o protocolo

A primeira coisa escrita não foi código: foi a **tabela de comandos**. Antes de saber como o servo seria movido, foi definido exatamente o que atravessa o cabo. Isso fixa o contrato entre os dois lados e permite que Python e firmware sejam escritos e testados separadamente.

HOST → MICROCONTROLADOR	SIGNIFICADO
G:<polegar>,<indicador>,<outros>	ângulos absolutos em graus; sempre passa pelo clamp
P:<nome>	gesto nomeado: OPEN FIST POINT PINCH SHAKE
S	stop seguro — abre a mão e larga o objeto
H	heartbeat: “o host está vivo”
?	pede o status atual
MICROCONTROLADOR → HOST	SIGNIFICADO
R	banner de boot: “acabei de ligar, estou pronto”
A:<eco>	aceito (ex.: A:G , A:P:FIST)
E:<cod>:<msg>	erro — 1 = fora de faixa, 2 = parse, 3 = watchdog, 4 = comando
S:<t>,<i>,<o>,<modo>	status; modo é HOST ou SAFE

Por que ASCII, e não bytes binários

Um protocolo binário seria mais compacto. Foi escolhido texto por três razões práticas, e todas as três se pagaram no laboratório:

1. **Você pode digitar à mão.** Com um monitor serial aberto, dá para digitar **P:FIST** e ver a mão fechar. Não precisa de programa nenhum para testar o hardware.
2. **Um humano lê o log.** Quando algo dá errado, a serial mostra **E:2:parse** , não **0x02** . O diagnóstico da Parte 6 só existe porque a conversa é legível.
3. **Não precisa de framing nem checksum.** O **\n** já delimita a mensagem, e a linha é curta o bastante para que corrupção seja rara e detectada pelo parse.

Uma sessão real, digitada à mão no monitor serial — o que você escreve está marcado, o resto é o firmware respondendo:

```

R                ← a placa acabou de ligar
?
S:15,15,15,HOST  ← mão aberta, sob controle do host
P:FIST
A:P:FIST         ← aceito; a mão começa a fechar suavemente
G:200,30,30
E:1:range        ← 200° recusado... mas o clamp aplicou 165°
G:165,30,30
A:G
S
A:S              ← stop: abriu a mão
```

O protocolo é fonte de verdade compartilhada: qualquer mudança aqui precisa entrar também em `src/thoth/actuation/serial_client.py` . Está documentado em `docs/protocol.md` e no cabeçalho do próprio sketch.

Anatomia do `loop()`, na ordem de execução

Todo o firmware cabe em três tarefas que se repetem. A ordem importa: primeiro escuta, depois verifica se ainda há alguém do outro lado, e só então move o motor.



o passo 3 tem um portão de tempo; os passos 1 e 2 rodam em TODA volta

A tarefa 3 é a única com portão de tempo. As tarefas 1 e 2 rodam sempre — é isso que faz o fail-safe ser rápido e o comando nunca ser perdido.

```

void loop() {
    unsigned long now = millis();

    // 1) leitura serial não-bloqueante, linha a linha
    while (Serial.available()) {
        char c = (char)Serial.read();
        if (c == '\n' || c == '\r') { lineBuf[lineLen] = '\0'; handleLine(lineBuf); lineLen = 0; }
        else if (lineLen < sizeof(lineBuf) - 1) { lineBuf[lineLen++] = c; }
        else { lineLen = 0; Serial.println(F("E:2:parse")); } // overflow → descarta a linha
    }

    // 2) watchdog de heartbeat → fail-safe = abrir a mão
    if (!safeMode && (now - lastHeartbeat > WDT_MS)) { goSafeOpen(); Serial.println(F("E:3:wdt")); }

    // 3) tick de controle a 50 Hz
    if (now - lastTick >= TICK_MS) { lastTick = now; controlTick(); }
}
  
```

Trinta linhas. Não há interrupções, não há threads, não há alocação dinâmica — e é justamente por isso que o comportamento é previsível a cada volta.

Seis decisões de baixo nível, e o porquê de cada uma

Programar embarcado é menos sobre sintaxe e mais sobre um conjunto de perguntas que se fazem por reflexo. Um iniciante escreve o código que funciona no caso bom; essas perguntas são o que transforma isso em firmware.

AS CINCO PERGUNTAS QUE PRODUZIRAM ESTE FIRMWARE

1. Isso bloqueia o laço? · 2. O que acontece se o outro lado morrer — e qual é o estado *seguro*? · 3. Quem valida esse número? · 4. O que a mecânica faz quando eu solto o sinal? · 5. Isso atrapalha a próxima regravação?

01 Parser byte a byte, com buffer e proteção de overflow

PROBLEMA	Os bytes chegam picados: metade da linha pode chegar numa volta do laço e o resto na seguinte. Ler “uma linha” não é uma operação atômica.
DECISÃO	Acumular caracteres em <code>lineBuf[48]</code> e só chamar <code>handleLine()</code> ao encontrar <code>\n</code> . Se a linha passar de 47 caracteres, descartar tudo e responder <code>E:2:parse</code> .
SEM ISSO	Comandos partidos ao meio seriam interpretados como lixo, e um byte perdido dessincronizaria o canal para sempre.

02 Alvo separado do atual: slew-rate a 50 Hz

PROBLEMA	Mandar <code>servo.write(165)</code> de uma vez faz o servo sair com toda a velocidade e bater no fim de curso. Isso arranca dente de engrenagem e quebra peça impressa.
DECISÃO	Guardar dois valores por dedo — <code>tgt</code> (para onde quero ir) e <code>cur</code> (onde estou) — e, a cada 20 ms, aproximar <code>cur</code> de <code>tgt</code> por no máximo <code>STEP_BIG = 10°</code> (ou <code>STEP_THUMB = 7°</code> no polegar).
EFEITO	O movimento fica contínuo e com aparência intencional, e a velocidade máxima passa a ser uma constante ajustável em vez de uma característica do motor.

```
if (curIndex != tgtIndex)
    curIndex += constrain(tgtIndex - curIndex, -STEP_BIG, STEP_BIG);    // no máximo 10° por tick
```

03 Clamp: o firmware nunca confia no host

PROBLEMA	Um bug em Python, um landmark ruim da câmera ou um comando digitado errado podem pedir 400°. O servo tentaria alcançar e forçaria contra o batente.
DECISÃO	Todo ângulo que entra passa por <code>constrain(a, 15, 165)</code> dentro do firmware. Se o valor original estava fora da faixa, responde <code>E:1:range</code> — e ainda assim aplica o valor limitado, porque recusar o movimento inteiro é pior que limitá-lo.
PRINCÍPIO	O limite mais próximo do músculo é o que vale. A validação em Python é conveniência; a do firmware é segurança.

Os mesmos limites existem espelhados em `src/thoth/safety/limits.py`, cuja primeira linha declara ser a *fonte única de verdade* e nomeia as constantes do firmware que precisa acompanhar.

04 Fail-safe por heartbeat — e por que o estado seguro é *abrir*

PROBLEMA	Se o Python travar, o cabo cair ou o notebook hibernar com um objeto na mão, o firmware não fica sabendo. Ele continuaria segurando indefinidamente.
DECISÃO	O host manda <code>H</code> a cada 0,3 s. Se passar <code>WDT_MS = 1000</code> ms sem heartbeat, o firmware abre a mão sozinho, entra em modo <code>SAFE</code> e avisa <code>E:3:wdt</code> . Qualquer comando válido volta ao modo <code>HOST</code> .
POR QUE ABRIR	Numa prótese, congelar é a falha perigosa: mantém a força sobre o objeto — ou sobre um dedo — sem ninguém no controle. Abrir <i>larga</i> . É a única escolha defensável, e ela é uma decisão de projeto, não de código.

05 Watchdog de software em vez do watchdog de hardware

PROBLEMA	O AVR tem um watchdog em hardware (<code>wdt_enable</code>), que parece a escolha óbvia. No clone com bootloader antigo ele travava a gravação (avalanche de <code>not in sync</code>) e podia deixar a placa em bootloop.
DECISÃO	Desligar o watchdog de hardware na primeira linha do <code>setup()</code> e mover toda a segurança para o heartbeat da decisão 04.
E O GANHO	O heartbeat mede a coisa certa. O watchdog de hardware detecta “o <i>firmware</i> travou”; o heartbeat detecta “o <i>host</i> parou de falar” — que é o risco real deste sistema.

```
void setup() {
  MCUSR = 0;
  wdt_disable();          // sem isso, a próxima gravação pode falhar ou entrar em bootloop
  Serial.begin(115200);
  curThumb = tgtThumb = THUMB_MIN;  // posição inicial segura = mão aberta
  attachAll();
  Serial.println(F("R"));           // banner de ready — a evidência da Parte 6
}
```

06 Ângulo lógico vs. físico, e o servo que fica energizado

PROBLEMA A	Do jeito que os servos estão montados, girar “para mais” fecha em um e abre em outro. Remontar mecanicamente daria trabalho e poderia ser desfeito por qualquer manutenção.
DECISÃO A	Definir uma convenção lógica única — menor = aberto, maior = fechado — e traduzir para o mundo físico na última linha antes de escrever, com um flag por servo.
PROBLEMA B	Os dedos da HACKberry têm mola de torção. Quando o PWM é cortado (<code>detach</code>), o servo deixa de segurar e a mola vence: o dedo volta sozinho, segundos depois do gesto.
DECISÃO B	<code>DETACH_WHEN_IDLE = false</code> . O servo fica energizado mantendo posição contra a mola — o que exige a fonte externa de 5–6 V, porque holding torque consome corrente.

```
int outAngle(int logical, bool rev) { return rev ? (180 - logical) : logical; }  
// toda a inversão de sentido do projeto cabe nesta linha  
  
const bool DETACH_WHEN_IDLE = false; // a mola de torção venceria o servo sem PWM
```

O QUE ESTE PAR DE DECISÕES ENSINA

Firmware é onde a mecânica e a eletrônica aparecem como código. A mola não é um detalhe de montagem: ela é um requisito que muda uma constante. E o problema mecânico mais real do projeto — torque de mola contra torque de retenção — foi resolvido como um problema de controle, em uma linha.

PARTE 5

O fluxo no VS Code, do zero ao servo girando

Este projeto não usa a IDE do Arduino. O `.ino` é editado no VS Code como qualquer arquivo de texto, e compilar e gravar são comandos de terminal — o mesmo terminal onde o lado Python roda. Quatro razões:

1. **Reprodutível.** A gravação é um comando escrito, não uma sequência de cliques que alguém precisa lembrar.
2. **Scriptável.** Vira `scripts/flash_firmware.py`, que tenta o bootloader certo primeiro e instala o que falta sozinho.
3. **Versionável.** Firmware, backend e visão vivem no mesmo repositório, no mesmo commit.
4. **Um só ambiente.** Editar o firmware, subir o painel web e ler a serial acontecem em abas do mesmo editor.

Passo 0 — instalar, uma vez só

```
winget install -e --id ArduinoSA.CLI # o compilador e o gravador  
arduino-cli core update-index  
arduino-cli core install arduino:avr # suporte ao ATmega328 (Nano)  
arduino-cli lib install Servo # a única biblioteca do firmware
```

No Windows, o `winget` adiciona o programa ao PATH da *máquina*: um terminal já aberto não enxerga. Feche e reabra o terminal do VS Code. (O script do projeto contorna isso procurando o executável nos caminhos conhecidos.)

Passo 1 — a regra de pasta que o Arduino exige

Não é convenção, é obrigatório: **cada sketch mora numa pasta com exatamente o mesmo nome do arquivo `.ino`**. Compilar aponta para a pasta, não para o arquivo.

```
firmware/
├─ hackberry_serial/
│   └─ hackberry_serial.ino    ← pasta e arquivo com o mesmo nome
└─ servo_scan/
    └─ servo_scan.ino         o sketch de diagnóstico (Parte 6)
```

Passo 2 — descobrir em que porta a placa está

```
python scripts/check_devices.py          # lista portas seriais e câmeras
# ou, direto pelo arduino-cli:
arduino-cli board list
```

No nosso setup o clone CH340 apareceu em `COM16`. Se nenhuma porta aparecer: o cabo é de dados e não só de carga? O driver CH340 está instalado?

Passo 3 — compilar e gravar

A forma recomendada é o script do projeto, que já tenta o bootloader antigo primeiro:

```
python scripts/flash_firmware.py --port COM16

# com o just instalado, o mesmo em menos teclas:
just flash COM16
```

FLAG	EFEITO
<code>--port COM16</code>	porta serial da placa (obrigatória)
<code>--old-bootloader</code>	força só o bootloader antigo (<code>cpu=atmega328old</code>)
<code>--new-bootloader</code>	força só o bootloader novo (Nano original)
<code>--sketch firmware/servo_scan</code>	grava outro sketch em vez do firmware principal

Por baixo, são dois comandos do `arduino-cli`. Vale saber fazer à mão, porque é o que você digita quando o script não está disponível:

```
# bootloader ANTIGO - o do clone CH340; tente este primeiro
arduino-cli compile --fqbn arduino:avr:nano:cpu=atmega328old firmware/hackberry_serial
arduino-cli upload --fqbn arduino:avr:nano:cpu=atmega328old -p COM16 firmware/hackberry_serial

# bootloader NOVO - Nano original (fallback)
arduino-cli compile --fqbn arduino:avr:nano firmware/hackberry_serial
arduino-cli upload --fqbn arduino:avr:nano -p COM16 firmware/hackberry_serial
```


O QUE É UMA FQBN, E POR QUE ELA MUDA TUDO

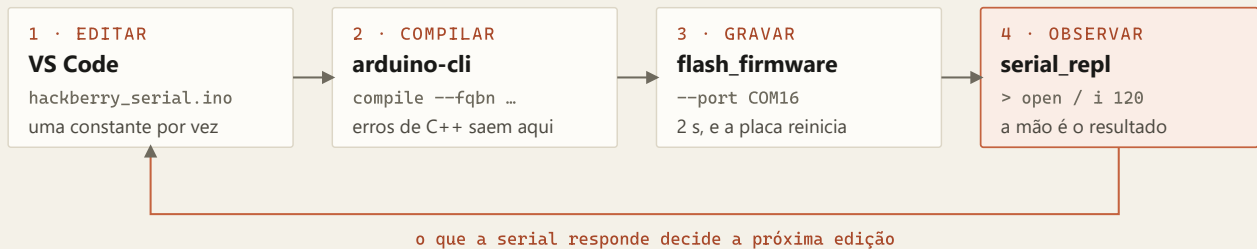
FQBN = *Fully Qualified Board Name*. É a identidade completa do alvo: `arduino:avr:nano` diz “fabricante Arduino, arquitetura AVR, placa Nano”, e `:cpu=atmega328old` acrescenta a variante de bootloader. Essa variante troca a velocidade com que o gravador conversa com a placa — 57600 bps em vez de 115200. Errar isso não dá erro de compilação: dá dez avisos de `not in sync` e nenhuma gravação.

Passo 4 — ver a placa falando

```
# monitor cru: mostra o que o firmware imprime, e você digita comandos
arduino-cli monitor -p COM16 -c baudrate=115200

# ou o REPL do projeto, que já fala o protocolo
python scripts/serial_repl.py --port COM16

> open           # gestos: open fist point pinch shake stop status
> i 120          # um dedo por vez: t/i/o + ângulo (polegar/indicador/três)
> i+            # ajuste fino de ±10° — é assim que se calibra
> G:165,15,15    # linha crua do protocolo, direto ao firmware
```



atenção: feche o monitor antes de gravar — a porta serial é de uso exclusivo

O ciclo inteiro dura menos de um minuto. É essa velocidade que permite calibrar por medição em vez de por cálculo.

Os cinco erros que todo mundo comete

SINTOMA	CAUSA	SOLUÇÃO
<code>not in sync</code> / <code>programmer not responding</code>	clone CH340 usa o bootloader antigo	grave com <code>cpu=atmega328old</code> (o script já tenta primeiro); confirme que o cabo é de dados
<code>Access is denied</code> ao gravar	o monitor serial está aberto e ocupando a porta	feche o monitor (ou o painel web) antes de gravar
a porta COM não aparece	driver CH340 ausente, ou cabo só de carga	instale o driver CH340; troque o cabo
<code>Servo.h: No such file</code>	biblioteca não instalada no <code>arduino-cli</code>	<code>arduino-cli lib install Servo</code>
compila e grava, mas o servo treme ou não segura	não é software: é corrente	fonte externa 5–6 V ≥ 2 A só para os servos, com GND comum

Sobre IntelliSense: a extensão C/C++ do VS Code dá realce e navegação, mas não conhece os cabeçalhos do Arduino sem configuração. Neste projeto a fonte de verdade para erro é o `arduino-cli compile` no terminal — sublinhado vermelho no editor sobre `Servo.h` não significa que não compila.

Como se prova que funcionou

A parte mais difícil de baixo nível não é escrever o código — é descobrir por que o mundo físico não está de acordo com ele. Três hábitos fizeram a diferença neste projeto, e valem para qualquer bring-up.

A Um instrumento por dúvida, em vez de um palpite melhor

Só dois dos três servos respondiam, e a pinagem assumida estava errada. Em vez de tentar outra combinação, foi escrito um segundo sketch de 44 linhas cuja única função é medir: ele varre D2 a D12, move o servo que estiver em cada pino e *anuncia na serial qual pino está testando*. Você olha a mão e anota.

```
python scripts/flash_firmware.py --port COM16 --sketch firmware/servo_scan
arduino-cli monitor -p COM16 -c baudrate=115200
```

```
=== SERVO SCAN: observe qual dedo se move em cada pino ===
>>> Testando pino D2
>>> Testando pino D3      ← o indicador mexeu aqui
>>> Testando pino D4      ← o polegar
>>> Testando pino D5      ← os três dedos
```

O resultado — `PIN_INDEX = 3`, `PIN_THUMB = 4`, `PIN_OTHER = 5` — virou constante do firmware. Trocamos adivinhação por medição.

B Ler a serial antes de teorizar — inclusive o que *não* aparece

O servo tremia e não segurava. Podia ser mil coisas. A serial resolveu em um passo: o banner `R`, que só é impresso no boot, **não reaparecia**. Se a placa não está reiniciando, o problema não é crash nem bootloop — não é software. Logo, é corrente. E era: três servos sob carga puxam de 1 a 3 A, e a porta USB entrega cerca de 500 mA.

A LIÇÃO TRANSFERÍVEL

Uma ausência no log é um dado, não a falta de um dado. Um `Serial.println` colocado no lugar certo — o banner de boot — vale mais que dez hipóteses, porque ele divide o espaço de causas em dois.

C O hardware é o juiz

Nenhuma decisão desta lista foi aceita porque parecia certa. Cada uma virou uma constante, foi gravada e foi ao servo. Se a hipótese estivesse errada, o dedo continuava voltando sozinho ou o servo continuava tremendo. Em baixo nível a verificação é externa e não admite argumento — e é isso que torna o aprendizado rápido.

Cartão de referência

PRECISO...	COMANDO
descobrir a porta e a câmera	<code>python scripts/check_devices.py</code>
gravar o firmware	<code>python scripts/flash_firmware.py --port COM16</code>
gravar o diagnóstico de pinos	<code>... --sketch firmware/servo_scan</code>
ver a placa falando	<code>arduino-cli monitor -p COM16 -c baudrate=115200</code>
controlar e calibrar à mão	<code>python scripts/serial_repl.py --port COM16</code>
subir o painel web	<code>python scripts/web.py</code>
CONSTANTE	VALOR E POR QUÊ
<code>PIN_THUMB / PIN_INDEX / PIN_OTHER</code>	4 / 3 / 5 — medidos com o <code>servo_scan</code>
<code>THUMB/INDEX/OTHER_MIN ... MAX</code>	15 ... 165° — curso útil calibrado dedo por dedo
<code>STEP_BIG / STEP_THUMB</code>	10° / 7° por tick — limita a velocidade, protege a mecânica
<code>TICK_MS</code>	20 ms → laço de controle a 50 Hz
<code>WDT_MS</code>	1000 ms sem heartbeat → abre a mão
<code>DETACH_WHEN_IDLE</code>	<code>false</code> — a mola de torção venceria o servo sem PWM
<code>REV_THUMB / REV_INDEX / REV_OTHER</code>	<code>true</code> — os três servos giram ao contrário nesta montagem

Firmware sob **GPLv3**, por derivar do sketch HACKberry de *exiii Inc. / Mission ARM Japan*. O hardware HACKberry é CC BY-NC-SA 4.0 (uso não comercial). Código, guia de montagem e guia de replicação com lista de materiais estão no repositório do Projeto Thoth.